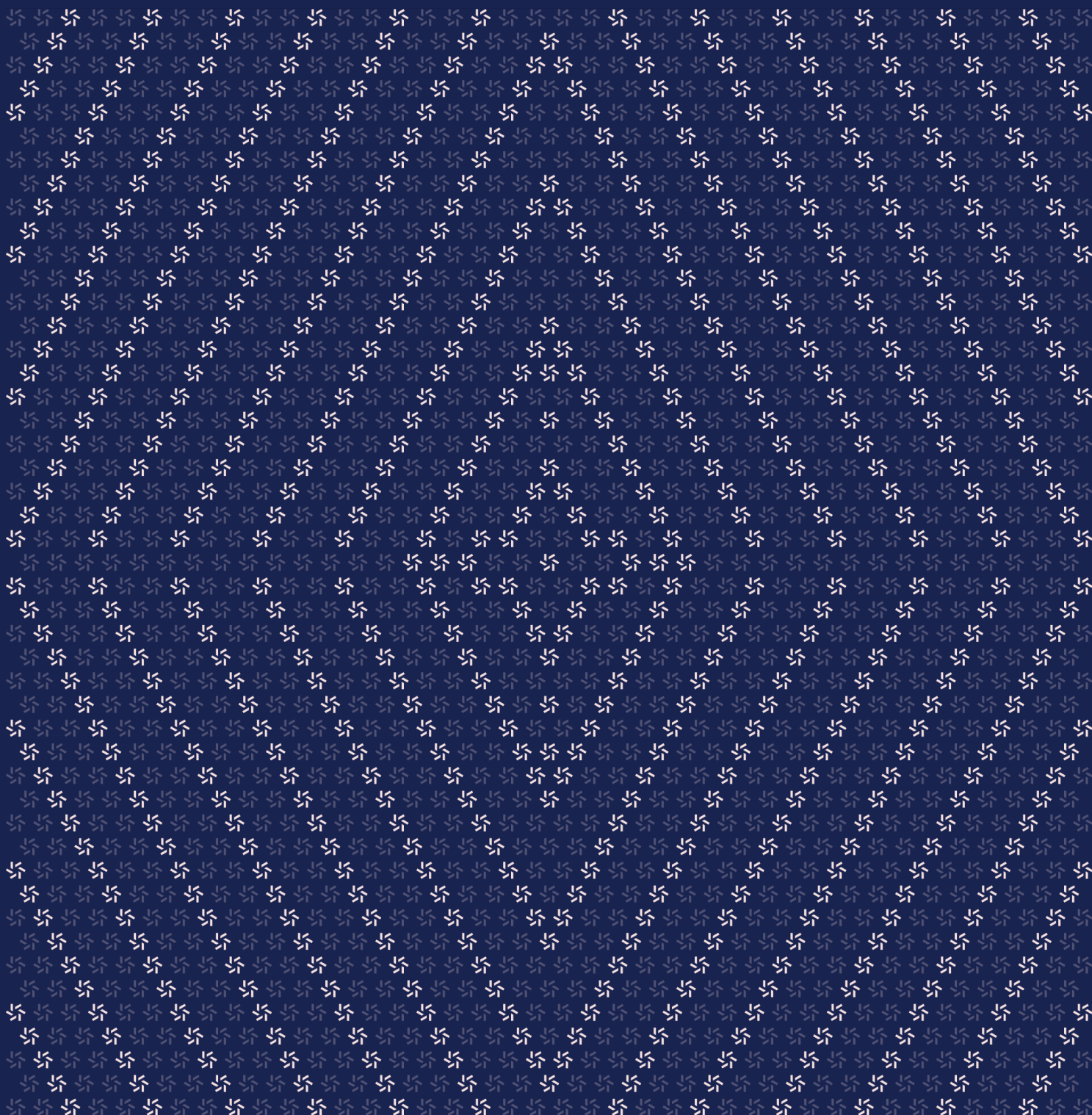


June 22, 2026

FIP 16

Smart Contract Patch Review



Contents

About Zellic	3
1. Introduction	3
1.1. Scope	4
1.2. Disclaimer	6
2. Detailed Findings	6
2.1. Per-voter WNat cap can be bypassed by splitting across controlled voters	7
2.2. Governance updates retroactively alter signing-policy burn penalties	10
2.3. Ensure atomic Inflation state migration and activation	12
2.4. Inflation base will not use foundationAddresses balance after FIP-16	14
2.5. Redeploying WNatDelegationFee resets voter delegation fee settings	16
3. Patch Review	16
3.1. Notable changes	17
3.2. Minor changes	17

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Introduction

We were asked to review the smart-contract changes introduced for Flare Improvement Proposal 16 (FIP 16 [↗](#)) across two of Flare's core protocol repositories. The review was conducted from June 17th to June 18th, 2026.

The following is a brief summary of the changes introduced across the two repositories; a more detailed description is provided in Section [3](#) [↗](#):

- In `flare-smart-contracts-v2` the change touches FTSO voter weighting and fees. `FlareSystemsCalculator` gains a governance-settable `stakingFactor` (`uint16`) multiplier that is applied to node staking weights when computing a voter's registration weight: the registration weight is now an approximation of $(\text{stakingFactor} * \text{staking weight} + \text{capped WNat weight})$ to the power of 0.75, where before the staking weight entered with an implicit factor of one. `StakingFactor` is set in the constructor and via `setStakingFactor`, which is restricted to governance and only callable while voter registration is disabled. The same diff also adds a governance `setStakingFactor` guard, a `setSigningPolicySignDurations` governance setter, additional configuration events, converts require strings to custom errors, and bumps the pragma from 0.8.20 to 0.8.27. `WNat-DelegationFee` gains an immutable `minFeeBIPS` minimum fee: `setVoterFeePercentage` now requires the fee to be at least `minFeeBIPS` (and at most `MAX_BIPS`), and the constructor enforces `minFeeBIPS <= MAX_BIPS` and `defaultFeePercentageBIPS >= minFeeBIPS`.
- In `flare-smart-contracts-v1` the change touches inflation recognition. Inflation now recognizes inflation against circulating supply (`supply.getCirculatingSupplyAt`) instead of the legacy inflatable balance; `Supply.getInflatableBalance` is marked deprecated and kept only for backward compatibility. The previously hardcoded `MAX_ANNUAL_INFLATION` constant in `InflationTimeSlots` is removed and replaced by a governance-settable `maxAnnualInflationWei` state variable on `Inflation`, set in the constructor and via `setMaxAnnualInflation` (governance only, require 0), and threaded into `_computeRecognizedInflationWei` to cap monthly recognized inflation at `maxAnnualInflationWei / 12`. The obsolete legacy interface `IIInflationV1.sol` is deleted.

1.1. Scope

The engagement involved a review of the following targets:

FIP 16 Contracts

Type	Solidity
Platform	EVM-compatible

Target	flare-smart-contracts-v1
Repository	https://github.com/flare-foundation/flare-smart-contracts-v1 ↗
Version	Diff between commit 10b4909f0ef58fc70ae7af8e2e5fad4180f42be8 and 9be10b4fde7a7187cef0586f06c6ffdf94466eff
Programs	contracts/inflation/implementation/Inflation.sol contracts/inflation/implementation/Supply.sol contracts/inflation/lib/InflationTimeSlots.sol contracts/inflation/interface/IIInflationV1.sol
Target	flare-smart-contracts-v2
Repository	https://github.com/flare-foundation/flare-smart-contracts-v2 ↗
Version	Diff between commit 264dab74ed0cef9c03a46596908978d954002338 and 62aca6a6ec0fa59b784526c67f40883e787aba96
Programs	contracts/protocol/implementation/FlareSystemsCalculator.sol contracts/protocol/implementation/WNatDelegationFee.sol contracts/userInterfaces/IFlareSystemsCalculator.sol contracts/userInterfaces/IWNatDelegationFee.sol

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
✧ Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Nipun Gupta
✧ Engineer
nipun@zellic.io ↗

Weipeng Lai
✧ Engineer
weipeng.lai@zellic.io ↗

1.2. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.

2. Detailed Findings

2.1. Per-voter WNat cap can be bypassed by splitting across controlled voters

Target	FlareSystemsCalculator		
Category	Business Logic	Severity	Medium
Likelihood	Low	Impact	Medium

Description

The `calculateRegistrationWeight()` function in `FlareSystemsCalculator` calculates each voter's registration weight independently. For each `_voter`, it fetches the voter's delegation address, reads that address's WNat vote power, caps it at `wNatCapPPM` of total WNat vote power, and then applies the 0.75 exponent to the combined registration weight.

```
function calculateRegistrationWeight(
    address _voter,
    uint32 _rewardEpochId,
    uint256 _votePowerBlockNumber
)
external onlyVoterRegistry
returns (uint256 _registrationWeight)
{
    // [...]
    uint256 wNatWeight = 0;
    uint256 wNatCappedWeight = 0;
    address delegationAddress = entityManager.getDelegationAddressOfAt(_voter,
        _votePowerBlockNumber);
    if (_rewardEpochId
    >= voterRegistry.chilledUntilRewardEpochId(bytes20(delegationAddress))) {
        uint256 totalWNatVotePower
        = wNat.totalVotePowerAt(_votePowerBlockNumber);
        uint256 wNatWeightCap = (totalWNatVotePower * wNatCapPPM) / PPM_MAX;
        // no overflow possible
        wNatWeight = wNat.votePowerOfAt(delegationAddress,
            _votePowerBlockNumber);
        wNatCappedWeight = Math.min(wNatWeightCap, wNatWeight);
        _registrationWeight += wNatCappedWeight;
    }
    uint16 delegationFeeBIPS = wNatDelegationFee.getVoterFeePercentage(_voter,
        _rewardEpochId);

    _registrationWeight = _sqrt(_registrationWeight);
    _registrationWeight *= _sqrt(_registrationWeight);
}
```

```
// [...]  
}
```

VoterRegistry then stores each voter's calculated weight separately and later normalizes all registered voter weights into signing-policy weights. EntityManager enforces uniqueness of delegation, submit, submit-signatures, and signing-policy addresses, but it does not aggregate multiple voters that are controlled by the same WNat holder or operator.

As a result, the WNat cap and 0.75 exponent are enforced per voter identity, not per economic owner. A single WNat holder can split their own WNat voting power across multiple controlled delegation addresses, register each corresponding voter, and receive the sum of several independently capped and exponentiated weights.

This creates two effects:

- **Cap bypass:** the 2.5% WNat cap applies per voter. A holder with more than the cap can split across multiple voters and have more of their WNat counted in aggregate than would be counted through one voter.
- **Concavity amplification:** because $x^{0.75}$ is concave, $\sum((x_i)^{0.75}) > (\sum(x_i))^{0.75}$. Splitting the same total WNat into multiple registered voters increases the aggregate calculated weight.

Impact

A well-capitalized operator can increase their aggregate signing-policy weight and reward share without adding proportional WNat economic weight. Because rewards are distributed according to normalized weights, the attacker's increased share dilutes the reward share of other stakers and delegators.

The attack can also bypass the intended effect of the WNat cap: although any individual voter is capped, a cluster of voters controlled by the same actor can exceed the cap in aggregate. This weakens the cap as a concentration-control mechanism.

The impact is bounded by practical constraints:

- The amplification is not free. Each registered identity must satisfy Flare's [minimum participation requirements](#) to remain reward-eligible, including staking/self-bond requirements, uptime, FTSO participation, FDC participation, and pass mechanics.
- VoterRegistry only registers up to the configured maxVoters; once full, a new voter can displace an existing lower-weight voter.
- The owner can only split WNat they themselves own; vote power delegated to them by third parties cannot be multiplied.

Recommendations

There is no fully reliable on-chain way to identify common economic ownership, so complete prevention is difficult. The protocol should explicitly decide whether the WNat cap is intended to be per registered provider or per economic owner.

- If per-provider capping is intended, document this as an accepted design tradeoff, including the fact that a single operator can exceed the cap in aggregate by running multiple providers.
- If per-owner concentration is meant to be limited, consider use off-chain detection and governance processes to identify clusters of commonly controlled providers and apply chilling or other penalties where abuse is detected.

Remediation

This issue has been acknowledged by Flare Foundation.

Flare Foundation provided the following response to this finding:

It is desirable for larger entities to run multiple data provider instances, provided they also operate separate infrastructure for each which improves price and client diversity. Any potential collusion is closely monitored.

2.2. Governance updates retroactively alter signing-policy burn penalties

Target	FlareSystemsCalculator		
Category	Business Logic	Severity	Low
Likelihood	Low	Impact	Low

Description

The newly added `setSigningPolicySignDurations()` function in `FlareSystemsCalculator` allows governance to modify the parameters used to calculate signing-policy burn penalties:

```
function setSigningPolicySignDurations(
    uint64 _signingPolicySignNonPunishableDurationSeconds,
    uint64 _signingPolicySignNonPunishableDurationBlocks,
    uint64 _signingPolicySignNoRewardsDurationBlocks
)
external onlyGovernance
{
    signingPolicySignNonPunishableDurationSeconds
    = _signingPolicySignNonPunishableDurationSeconds;
    signingPolicySignNonPunishableDurationBlocks
    = _signingPolicySignNonPunishableDurationBlocks;
    signingPolicySignNoRewardsDurationBlocks
    = _signingPolicySignNoRewardsDurationBlocks;
    emit SigningPolicySignDurationsSet(
        _signingPolicySignNonPunishableDurationSeconds,
        _signingPolicySignNonPunishableDurationBlocks,
        _signingPolicySignNoRewardsDurationBlocks
    );
}
```

`calculateBurnFactorPPM()` calculates the burn factor for a given reward epoch by combining historical signing data with these parameters. `FlareSystemsManager` records the signing period's start and end timestamps and block numbers, as well as each voter's signing block, on a per-epoch basis. However, the thresholds against which these values are evaluated are read directly from mutable storage and are not checkpointed per epoch.

`RewardManager` calls `calculateBurnFactorPPM()` only when processing a FEE claim. Because rewards remain claimable after an epoch has ended, governance can update the parameters while claims from earlier epochs remain unclaimed. Those claims are consequently evaluated using the parameters in effect at claim time, rather than those in effect during the relevant signing-policy period.

Therefore, calling `setSigningPolicySignDurations()` can retroactively change the burn penalties applied to current or completed reward epochs. Before this function was introduced, the parameters were set only during construction and were effectively immutable, ensuring that historical burn calculations remained stable.

Impact

Identical signing behavior may result in different burn amounts depending solely on when the corresponding fee reward is claimed. A less punitive update may allow late-signing voters with unclaimed claims to retain rewards that would otherwise have been burned. Conversely, a more punitive update may increase the burn applied to users who have not yet claimed.

Recommendations

Apply signing-policy burn-parameter updates only to future reward epochs. Version each parameter set by an `effectiveFromRewardEpochId`, and have `calculateBurnFactorPPM()` use the version applicable to the claimed reward epoch.

Remediation

This issue has been acknowledged by Flare Foundation.

Flare Foundation provided the following response to this finding:

Any such governance change will be made only after data providers vote, giving them enough time to claim their rewards.

2.3. Ensure atomic Inflation state migration and activation

Target	Inflation		
Category	Business Logic	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The `setInitialData()` function in Inflation performs a one-time migration by reading the old Inflation contract's current state and copying its authorization timestamps, cumulative totals, reward-service accounting, top-up configurations, inflation time slots, and current time-slot pointer.

This creates a point-in-time snapshot but does not activate the new contract. Activation occurs separately through `AddressUpdater::update()`, which repoints `FlareDaemon` and other dependent contracts to the new Inflation address.

Until the address update executes, `FlareDaemon` continues to drive the old Inflation contract. Two paths can advance the state that `setInitialData()` copies:

1. `FlareDaemon::trigger()` calls the old `Inflation::daemonize()`. Its authorization and top-up request branch is gated by `lastAuthorizationTs + AUTHORIZE_TIME_FRAME_SEC <= block.timestamp` (`AUTHORIZE_TIME_FRAME_SEC = 1 days`), so it runs once per day, updating authorization timestamps, cumulative totals, and reward-service accounting, and requesting minting from `FlareDaemon`. `daemonize()` also recognizes a new inflation time slot when the current slot ends (approximately every 30 days), which updates the recognized totals and the time-slot collection.
2. In the block following a mint request, `FlareDaemon::trigger()` forwards the newly expected `MintRequest` minted amount to `Inflation::receiveMinting()`, which updates the top-up distribution accounting.

Outside of these events, intervening `daemonize()` calls do not change the copied accounting. However, if `setInitialData()` is called and the old contract later processes an inflation authorization, top-up distribution, or new time-slot initialization before activation, the old contract will advance beyond the copied snapshot and the new contract may start from stale state.

Impact

An incorrectly sequenced migration could cause the new Inflation contract's accounting to differ from the old contract's final state.

This is an operational migration concern rather than an externally exploitable contract vulnerability and can be avoided through the governance execution procedure.

Recommendations

Ensure that no relevant state change occurs on the old `Inflation` contract between the `setInitialData()` snapshot and activation. In particular, governance should ensure that there is no inflation authorization, no top-up distribution, and no new time-slot initialization during this interval.

The governance timelock should also be taken into account when sequencing the migration. In production, calling `AddressUpdater::update()` queues the update behind the timelock rather than executing it immediately, so the migration sequence must be coordinated accordingly.

Remediation

This issue has been acknowledged by Flare Foundation.

2.4. Inflation base will not use foundationAddresses balance after FIP-16

Target	Inflation		
Category	Business Logic	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The foundationAddresses balance were not subtracted in the getInflatableBalance calculation before fip 16. The inflatable balance was: initial genesis amount + total claimed - total excluded/locked amount:

```
function getInflatableBalance()
    external view override returns(uint256 _inflatableBalanceWei) {
        if (block.number < switchOverBlock) {
            return oldSupply.getInflatableBalance();
        }
        return initialGenesisAmountWei
            .add(totalClaimedWei)
            .sub(totalExcludedSupplyWei.sub(distributedExcludedSupplyWei))
            .sub(totalLockedWei);
    }
```

After the FIP-16 update as circulating supply is used in _initNewTimeSlot for inflation base and the _updateFoundationAddressesAmount subtracts foundation address balances from circulating supply, it will not contain foundation address balances.

```
function _initNewTimeSlot(uint256 startTs) internal {
    supply.updateCirculatingSupply();
    uint256 inflatableSupply = supply.getCirculatingSupplyAt(block.number);
    //...
}

function _updateCirculatingSupply() internal {
    //...

    _updateBurnAddressesAmount();
    _updateFoundationAddressesAmount();
}

function _updateFoundationAddressesAmount() internal {
```

```
uint256 newBalance = 0;
address[] memory addresses = foundationAddresses.list;
for (uint256 i = 0; i < addresses.length; i++) {
    newBalance = newBalance.add(addresses[i].balance);
}
if (foundationAddressesBalance == newBalance) {
    return;
} else if (foundationAddressesBalance > newBalance) {
    _increaseCirculatingSupply(foundationAddressesBalance - newBalance);
} else {
    _decreaseCirculatingSupply(newBalance - foundationAddressesBalance);
}
foundationAddressesBalance = newBalance;
}
```

Impact

The foundation addresses balance was previously included as a part of the inflation base calculation but after fip-16 it will not be included in that calculation. There is mention of this change in the [FIP-16 proposal](#) ↗

Recommendations

If this is the intended behaviour, we suggest mentioning the change in the proposal, else we recommend adding the foundation addresses balance back in the inflation calculation.

Remediation

This issue has been acknowledged by Flare Foundation.

Flare Foundation provided the following response to this finding:

As of now, there are no foundation addresses on the Flare chain. If this changes, we will notify the community.

2.5. Redeploying WNatDelegationFee resets voter delegation fee settings

Target	WNatDelegationFee		
Category	Business Logic	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

Voter fee histories are stored in the voterFeePercentages mapping within WNatDelegationFee. A newly deployed instance starts with empty storage and provides no mechanism for importing settings from the previous deployment.

Consequently, `_getVoterFeePercentage()` returns `defaultFeePercentageBIPS` for every voter without a setting in the new contract. Although the previous settings remain stored in the old contract, they are no longer consulted after the protocol switches to the new deployment.

Impact

Voters' custom delegation fees and scheduled fee changes will be lost during migration. All affected voters will temporarily use the protocol default fee until they re-submit their intended setting and the configured update offset has elapsed.

Recommendations

Notify voters before redeployment that they must re-apply their intended delegation fees using the new WNatDelegationFee contract. Clearly communicate that submitted settings become effective only after `feePercentageUpdateOffset` reward epochs.

Remediation

This issue has been acknowledged by Flare Foundation.

Flare Foundation provided the following response to this finding:

Almost all data providers have the default delegation fee; however, we will notify them once the contract is upgraded.

3. Patch Review

The purpose of this section is to document the diffs of the codebase that were considered in scope for this audit.

As requested by Flare Foundation, we focused on the changes made to the in-scope files between commit [264dab74](#) and commit [62aca6a6](#) in FSP V2, as well as the changes made to the in-scope files between commit [10b4909f](#) and commit [9be10b4f](#) in FSP V1.

3.1. Notable changes

The following are notable changes made to the codebase.

- **FSP V2 registration weights now apply a configurable staking factor.** `FlareSystem-sCalculator` now multiplies each non-chilled P-Chain node weight by the constructor-supplied `stakingFactor` before combining it with capped WNat vote power and applying the existing approximate three-quarter-power calculation. Governance can update the factor only while voter registration is disabled.
- **FSP V2 enforces a minimum WNat delegation fee.** `WNatDelegationFee` now stores an immutable `minFeeBIPS` value supplied at deployment. The constructor requires the minimum to be at most 100% and the default fee to fall between the minimum and 100%; voter-submitted fees must also remain within this range.
- **FSP V2 signing-policy burn parameters are now governance-configurable.** Governance can update the non-punishable duration in seconds, the non-punishable duration in blocks, and the additional block duration after which all rewards are burned through `setSigningPolicySignDurations()`. However, the update can retroactively alter signing-policy burn penalties (see finding [2.2](#)).
- **FSP V1 now recognizes inflation against circulating supply.** When initializing a new inflation time slot, `Inflation` updates the supply state and uses the current-block circulating supply instead of the inflatable balance as the percentage-based inflation base.
- **FSP V1 replaces the fixed annual inflation cap with a governance-controlled cap.** The hardcoded five-billion-token annual cap was removed from `InflationTimeSlots`. `Inflation` now receives a nonzero `maxAnnualInflationWei` value in its constructor, allows governance to update it, and passes it into the time-slot calculation, which caps monthly recognized inflation at one-twelfth of the configured annual amount.
- **FSP V1 migration preserves the current Inflation state.** `setInitialData()` was changed to a one-time migration from the currently deployed `Inflation` implementation. It verifies that both contracts use the same reward-epoch start, then copies authorization timestamps, cumulative totals, reward services, explicitly configured top-up settings, every existing time slot, and the current time-slot pointer. But note that atomic Inflation state migration and activation should be ensured (see finding [2.3](#)).

3.2. Minor changes

The following are minor changes made to the codebase.

- FlareSystemsCalculator now emits events when the WNat cap, signing-policy durations, or staking factor is set.
- Inflation now emits events when the maximum annual inflation is set and when initial state migration completes.
- Revert strings in FlareSystemsCalculator and WNatDelegationFee were replaced with custom errors declared in their public interfaces.
- The Solidity pragmas for FlareSystemsCalculator and WNatDelegationFee were raised from `^0.8.20` to `^0.8.27`.
- `Supply.getInflatableBalance()` remains available for backward compatibility but is now documented as deprecated because Inflation no longer uses it.